

Chương 16: Xử lý Ngôn ngữ Tự nhiên với RNN và Cơ chế Chú ý

Giảng viên

Đại học Đông Á

August 11, 2026

Mục tiêu của Chương

- Hiểu rõ và áp dụng RNN để xây dựng mô hình sinh văn bản tự động.
- Thực hành bài toán Phân tích cảm xúc (Sentiment Analysis) sử dụng Keras.
- Nắm vững kiến trúc mạng Encoder-Decoder dùng trong Dịch máy thần kinh (NMT).
- Phân tích chi tiết Cơ chế Chú ý (Attention Mechanisms) để khắc phục nhược điểm cổ chai của RNN.
- Tiếp cận và sử dụng siêu kiến trúc Transformer (BERT, GPT, ViT) thông qua thư viện Hugging Face.

Mục lục

1. Tạo văn bản kiểu Shakespeare bằng cách sử dụng RNN ký tự
2. Phân tích cảm xúc (Sentiment Analysis)
3. Mạng mã hóa-giải mã cho dịch máy thần kinh
4. Cơ chế chú ý (Attention Mechanisms)
5. Tuyệt lộ các mô hình Transformer & Hugging Face

1.1 Giới thiệu bài toán mô hình hóa ngôn ngữ (Language Modeling)

- **Khái niệm:** Mô hình hóa ngôn ngữ là nhiệm vụ dự đoán ký tự (hoặc từ) tiếp theo trong một chuỗi, dựa trên các ký tự (hoặc từ) trước đó.
- **Ứng dụng thực tế:**
 - Gợi ý từ khi gõ phím (Autosuggest).
 - Sửa lỗi chính tả tự động (Spell checking).
 - Sinh văn bản sáng tạo (Viết thơ, viết truyện, sinh code).
- **Vai trò của RNN:** Một mô hình RNN có khả năng duy trì trạng thái ẩn (hidden state) giúp "ghi nhớ" ngữ cảnh từ quá khứ, rất phù hợp cho dữ liệu văn bản tuần tự.

1.2 Mô hình hóa Cấp độ Ký tự (Character-level) vs Từ (Word-level)

Mô hình Character-level

- Dự đoán từng chữ cái một.
- Từ vựng nhỏ (vài chục ký tự).
- Xử lý tốt từ mới (Out-Of-Vocabulary) và lỗi chính tả.
- **Nhược điểm:** Chuỗi vào quá dài, mô hình dễ quên ngữ cảnh xa.

Mô hình Word-level

- Dự đoán từng từ một.
- Từ vựng rất lớn (hàng chục, hàng trăm ngàn từ).
- Nắm bắt ngữ nghĩa tốt hơn, chuỗi ngắn hơn.
- **Nhược điểm:** Khó xử lý từ hiếm gặp, kích thước ma trận Embedding khổng lồ.

1.3 Tập dữ liệu văn bản Shakespeare

- Trong ví dụ này, ta sử dụng toàn bộ tác phẩm của Shakespeare làm tập huấn luyện.
- Dữ liệu bao gồm các vở kịch, thơ, được gộp chung thành một file văn bản.
- Ta tải dữ liệu văn bản bằng Keras: `keras.utils.get_file()`.
- Văn bản chứa hơn 1 triệu ký tự, bao gồm các chữ cái (viết hoa/viết thường), dấu câu và khoảng trắng.

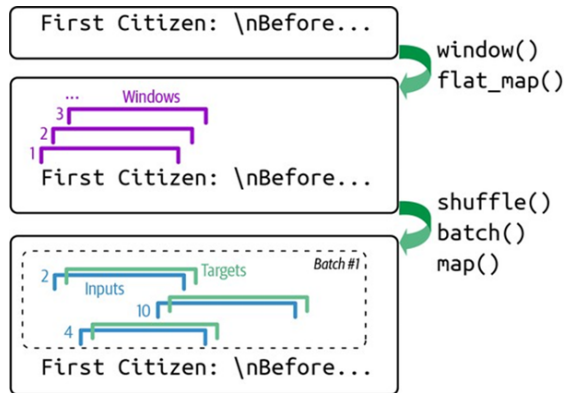
1.4 Tiền xử lý văn bản: Tokenizer trong Keras

- Mô hình nơ-ron không hiểu chữ cái, nó chỉ hiểu con số.
- Ta cần chuyển các ký tự thành số nguyên (ID) bằng Tokenizer.
- Keras hỗ trợ lớp Tokenizer cấu hình ở cấp độ ký tự.

```
from tensorflow import keras
# Khởi tạo Tokenizer cấp độ ký tự
tokenizer = keras.preprocessing.text.Tokenizer(char_level=True)
tokenizer.fit_on_texts(shakespeare_text)
# Chuỗi "First" -> [20, 6, 9, 8, 3]
max_id = len(tokenizer.word_index) # Tổng số ký tự duy nhất (~39)
dataset_size = tokenizer.document_count # Tổng ký tự (> 1 triệu)
```

1.5 Chuẩn bị dữ liệu bằng tf.data

- Ta tạo một cửa sổ (window) trượt qua văn bản để tạo ra các cặp (đầu vào, mục tiêu).
- Nếu chuỗi là "Thanh", window kích thước 4.
- Input: "Tha", Target: "h".
- Để tăng tốc, ta dùng `window(shift=1)` và batch dữ liệu.



Hình 16-1: Tóm tắt các bước chuẩn bị tập dữ liệu

1.6 One-hot Encoding so với Embedding

- **One-hot Encoding:** Biến số 5 thành véc-tơ $[0, 0, 0, 0, 1, 0, \dots]$.
 - Phù hợp khi bộ từ vựng rất nhỏ (39 ký tự).
 - Không mang thông tin ngữ nghĩa.
- **Embedding:** Biến số 5 thành véc-tơ số thực liên tục $[0.12, -0.5, 0.8, \dots]$.
 - Mô hình học được các biểu diễn ẩn tốt hơn.
 - Thường là lựa chọn tối ưu cho cả Word-level và Character-level hiện nay.

1.7 Xây dựng mô hình Char-RNN với GRU

- Mô hình Char-RNN gồm một lớp Embedding, các lớp GRU (hoặc LSTM) và một lớp Dense phân loại.
- `return_sequences=True` đảm bảo mỗi bước thời gian của lớp GRU trước đều đẩy kết quả lên lớp GRU sau.

```
model = keras.models.Sequential([
    keras.layers.Embedding(input_dim=num_chars, output_dim=16,
                           input_length=max_id),
    keras.layers.GRU(128, return_sequences=True, dropout=0.2),
    keras.layers.GRU(128, return_sequences=True, dropout=0.2),
    keras.layers.TimeDistributed(keras.layers.Dense(num_chars,
                                                       activation="softmax"))
])
```

1.8 Vấn đề Lặp văn bản khi Dự đoán (Argmax Loop)

- Nếu tại mỗi bước sinh ký tự mới, ta luôn chọn ký tự có xác suất cao nhất (**argmax**), văn bản tạo ra sẽ rất nhát nhẽo.
- Nguy hiểm hơn, mô hình có thể bị mắc kẹt trong vòng lặp vô tận (vd: "To be or not to be or not to be...").
- **Lý do:** Mô hình có xu hướng lặp lại các cụm từ phổ biến nhất và an toàn nhất trong tập dữ liệu.
- **Giải pháp:** Sử dụng việc **lấy mẫu ngẫu nhiên (Sampling)** dựa trên phân bố xác suất thay vì argmax.

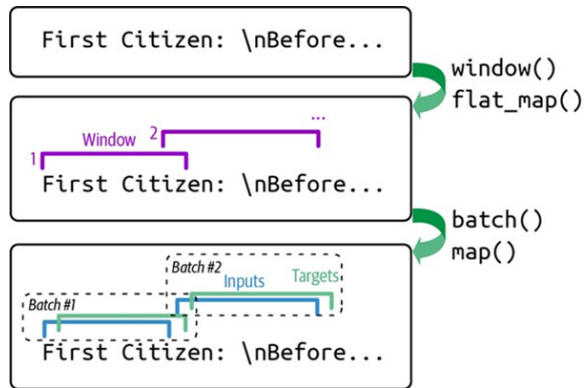
1.9 Sinh văn bản với Nhiệt độ (Temperature)

- Thay vì lấy logit trực tiếp, ta chia logit cho tham số **Temperature** (T) trước khi tính Softmax.
- $T \approx 0.1$: Chọn ký tự an toàn, chuẩn xác (giống argmax).
- $T \approx 1.0$: Lấy mẫu ngẫu nhiên bình thường.
- $T \approx 2.0$: Làm phẳng phân bố, tăng tính sáng tạo nhưng dễ sinh ra từ vô nghĩa.

```
import tensorflow as tf
# logits là mạng xác suất tu mô hình
rescaled_logits = logits / temperature
char_id = tf.random.categorical(rescaled_logits, num_samples=1)
```

2.1 Giới thiệu bài toán phân tích cảm xúc

- **Nhiệm vụ:** Phân loại văn bản thành cảm xúc "tích cực" (1) hoặc "tiêu cực" (0).
- **Tập dữ liệu:** **IMDb Reviews** chứa 50.000 bài đánh giá phim.
- **Thách thức:** Làm sao xử lý số lượng từ vựng khổng lồ (hàng chục ngàn từ) và biến chúng thành mảng số để đưa vào mạng?



Hình 16-2: Tóm tắt các bước xử lý dữ liệu IMDb

2.2 Xử lý dữ liệu ngôn ngữ (Word-level)

- **Tokenization:** Cắt câu thành các từ (tokens). Loại bỏ dấu câu và HTML tags (như `<br/>`).
- **Xây dựng từ điển (Vocabulary):** Ánh xạ mỗi từ thành một số ID nguyên (ví dụ: "the" -> 1, "movie" -> 2).
- **Cắt bớt từ vựng (Truncation):** Chỉ giữ lại 10.000 từ phổ biến nhất để giảm tải bộ nhớ và tính toán.
- Các từ hiếm (Out-Of-Vocabulary) sẽ được gán chung một ID đặc biệt là `<OOV>`.

2.3 Padding và Masking trong Keras

- **Padding:** Các bài đánh giá có độ dài khác nhau. Ta cần đệm thêm số 0 ở cuối (hoặc đầu) để đưa về cùng một độ dài max (vd: 500 từ).
- **Masking (mask_zero=True):** Mạng RNN phải tính toán theo thời gian. Nếu chuỗi có hàng trăm số 0 đệm, RNN sẽ học sai lệch. Việc bật Masking giúp Keras tự động bỏ qua các giá trị đệm 0 này.

```
keras.layers.Embedding(vocab_size, embed_size,  
                        input_shape=[None], mask_zero=True)
```

2.4 Kiến trúc LSTM cho Phân tích Cảm xúc

- Sử dụng Embedding để học biểu diễn từ liên tục.
- 2 lớp GRU (Gated Recurrent Unit) xếp chồng lên nhau giúp trích xuất ngữ cảnh dài hạn (ai làm gì, sắc thái chê hay khen).
- Lớp Dense cuối dùng sigmoid để ra xác suất 0-1.

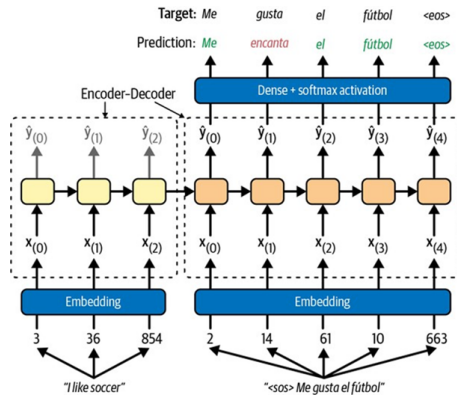
```
model = keras.models.Sequential([
    keras.layers.Embedding(vocab_size, embed_size, mask_zero=True),
    keras.layers.GRU(128, return_sequences=True),
    keras.layers.GRU(128),
    keras.layers.Dense(1, activation="sigmoid")
])
```


2.5 Đánh giá Kết quả và Hạn chế

- Với RNN/GRU/LSTM, độ chính xác trên tập IMDb có thể dễ dàng đạt **85% - 90%**.
- **Hạn chế 1 (Ngữ nghĩa):** Khó phân tích những bình luận có cấu trúc đảo ngữ phức tạp, dùng từ lóng, hoặc những câu mang tính mỉa mai (Sarcasm).
- **Hạn chế 2 (Tốc độ):** Mô hình có thể bị chậm do chuỗi dài (500 từ) và kiến trúc RNN không thể thực hiện tính toán song song.

3.1 Dịch máy thần kinh (Neural Machine Translation - NMT)

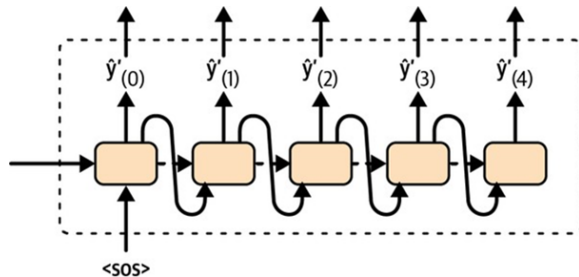
- Kiến trúc **Encoder-Decoder** là chuẩn mực cho các bài toán sequence-to-sequence (seq2seq).
- **Bộ mã hóa (Encoder)**: Đọc toàn bộ câu nguồn (ví dụ: tiếng Anh) và nén thành một véc-tơ trạng thái (Context Vector).
- **Bộ giải mã (Decoder)**: Nhận Context Vector làm điểm khởi đầu để sinh ra câu đích (ví dụ: tiếng Việt).



Hình 16-3: Mô hình Encoder-Decoder cơ bản

3.2 Cơ chế hoạt động của Decoder

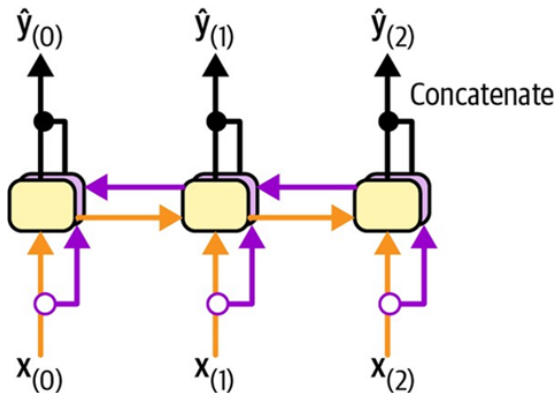
- Tại thời điểm t , Decoder lấy từ đã dự đoán ở bước $t - 1$ làm đầu vào cho bước tiếp theo.
- Nhưng trong khi huấn luyện, để tránh sai lầm dây chuyền, ta dùng **Teacher Forcing**: cung cấp từ mục tiêu thực tế (ground truth) làm đầu vào.
- Token `<sos>` báo hiệu bắt đầu dịch, và `<eos>` báo hiệu câu hoàn thành.



Hình 16-4: Chi tiết cấu trúc
Encoder-Decoder

3.3 Kiến trúc Encoder-Decoder Thực tế (Xếp chồng)

- Một mạng dịch thuật mạnh mẽ thường dùng nhiều lớp RNN/LSTM xếp chồng (VD: 2 đến 8 lớp).
- Trạng thái ẩn cuối cùng của toàn bộ các lớp trong Encoder sẽ được gán làm trạng thái ẩn khởi tạo cho các lớp tương ứng của Decoder.



Hình

16-5: Kiến trúc mạng NMT chi tiết với nhiều lớp LSTM

3.4 Kỹ thuật Tìm kiếm chùm (Beam Search)

- Nếu chỉ chọn từ có xác suất cao nhất ở mỗi bước (Greedy Search), nếu từ đầu tiên bị sai, toàn bộ câu sau sẽ hỏng.
- **Beam Search** khắc phục bằng cách theo dõi k chuỗi ứng viên (beam width) hứa hẹn nhất cùng lúc.
- Tại mỗi bước, nó mở rộng k chuỗi này và giữ lại top k chuỗi có xác suất tích lũy cao nhất để đi tiếp.



Hình 16-6: Minh họa Beam Search với $k=3$

3.5 Đo lường độ tốt của Dịch máy (BLEU Score)

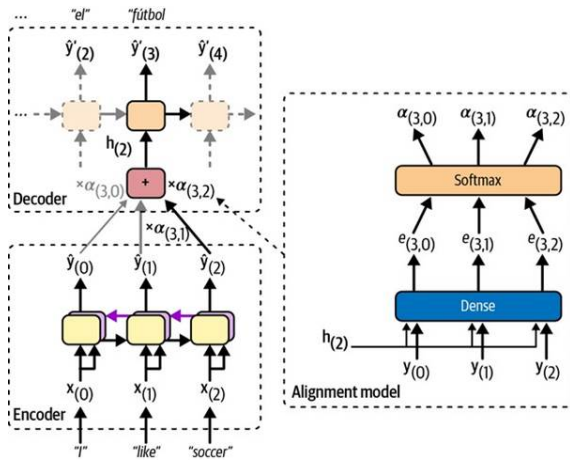
- Khác với phân loại văn bản, dịch máy không có độ chính xác (Accuracy) tuyệt đối vì một câu có thể được dịch theo nhiều cách đúng đắn.
- Thang đo chuẩn mực: **BLEU (Bilingual Evaluation Understudy)**.
- **Nguyên lý:** Đếm số lượng n-gram (từ hoặc cụm từ) xuất hiện trong bản dịch của máy có khớp với bản dịch tham chiếu của con người hay không.
- **Hình phạt (Penalty):** Trừ điểm nếu mô hình cố tình dịch câu quá ngắn để ăn gian xác suất khớp.

4.1 Vấn đề Nút thắt cổ chai (Bottleneck) của RNN

- Mô hình Encoder-Decoder truyền thống ép toàn bộ câu nguồn, dù dài hay ngắn, vào một **véc-tơ trạng thái tĩnh có kích thước cố định** (Context Vector).
- Khi câu nguồn rất dài (ví dụ: 50 từ), véc-tơ nhỏ bé này không đủ sức chứa mọi thông tin. Kết quả là mô hình thường "quên" phần đầu câu.
- Đây chính là hiện tượng nút thắt cổ chai thông tin.

4.2 Cơ chế Chú ý của Bahdanau (2014)

- **Giải pháp:** Thay vì bắt Decoder phải nhớ tất cả từ đầu, cho phép nó "nhìn lại" toàn bộ câu nguồn tại **mỗi bước dịch**.
- Tính toán **Trọng số chú ý**: Từ đích hiện tại có liên quan chặt chẽ đến từ nguồn nào nhất?
- Tạo ra Context Vector linh hoạt cho từng bước.



Hình

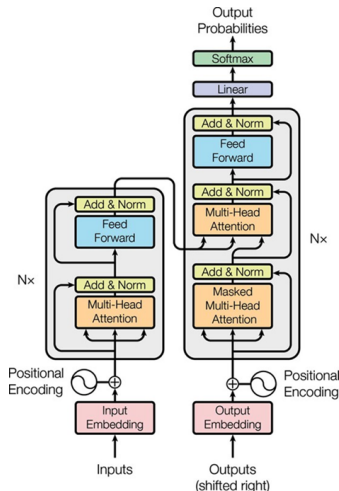
16-7: Encoder-Decoder với Cơ chế Chú ý

4.3 Sự Ra Đời Của Cú sốc "Transformer" (2017)

- Bài báo huyền thoại: **"Attention Is All You Need"** (Vaswani et al., Google, 2017).
- **Tuyên bố chấn động:** "Chúng tôi loại bỏ hoàn toàn cấu trúc RNN tuần tự. Chỉ sử dụng Attention thuần túy!"
- **Vì sao loại bỏ RNN?**
 - RNN xử lý tuần tự từng từ → Không thể song song hóa → Quá chậm trên GPU.
 - RNN vẫn bị giới hạn khi liên kết thông tin giữa từ thứ 1 và từ thứ 1000.
Transformer liên kết chúng trực tiếp bằng độ phức tạp $O(1)$.

4.4 Kiến trúc Transformer Gốc

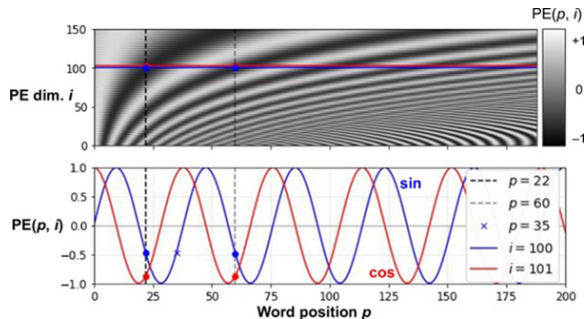
- Giữ lại ý tưởng Encoder-Decoder.
- Bên trong là hàng loạt khối **Multi-Head Attention** và mạng lan truyền xuôi (Feed Forward).
- Bổ sung thêm Skip Connections và Layer Normalization để chống suy biến gradient.
- Tốc độ huấn luyện tăng vọt nhờ GPU.



Hình 16-8: Kiến trúc siêu việt của Transformer

4.5 Mã hóa Vị trí (Positional Encoding)

- Transformer xử lý tất cả các từ cùng lúc (song song), nên nó mất đi khái niệm về **thứ tự từ**.
- Đối với Transformer, "Chó cắn mèo" và "Mèo cắn chó" là giống hệt nhau nếu không có thứ tự.
- **Giải pháp:** Cộng thêm một véc-tơ tạo bằng hàm Sin/Cosin vào mỗi từ để chỉ định vị trí của nó.



Hình 16-9: Hàm Sine và Cosine trong Positional Encoding

4.6 Cơ chế Tự Chú ý (Self-Attention)

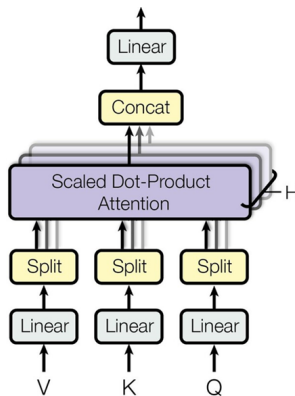
- Trong dịch máy trước đây, Decoder chú ý vào Encoder.
- **Self-Attention:** Một câu tự đánh giá mức độ liên quan giữa các từ bên trong chính nó.
- Ví dụ: "Ngân hàng (bank) nhà nước nằm bên bờ sông (bank)". Từ "bank" thứ nhất sẽ chú ý nhiều vào "nhà nước", còn "bank" thứ hai chú ý vào "bờ sông".
- Mọi từ đều được cập nhật ý nghĩa dựa trên toàn bộ ngữ cảnh câu.

4.7 Tính toán Q, K, V (Query, Key, Value)

- Self-Attention được mô hình hóa bằng cơ chế tìm kiếm trong cơ sở dữ liệu:
- **Query (Truy vấn):** Tôi là từ "ngân", tôi đang tìm kiếm cái gì?
- **Key (Khóa):** Tôi là từ "hàng", tôi có chứa thông tin gì?
- **Value (Giá trị):** Ý nghĩa thực sự của tôi là gì.
- Transformer dùng phép nhân ma trận giữa Q và K để đo độ tương đồng (Attention Score), sau đó dùng Score này để lấy trọng số cho V.

4.8 Chú ý Đa đầu (Multi-Head Attention)

- Một cái đầu (Head) không thể hiểu hết sự phức tạp của ngôn ngữ.
- **Multi-Head:** Phân tách phép tính Attention thành h "đầu" khác nhau (vd: 8, 12, hay 96 đầu).
- Mỗi đầu sẽ học cách tập trung vào một mối quan hệ riêng (vd: đầu 1 học ngữ pháp, đầu 2 học ý nghĩa, đầu 3 học đại từ nhân xưng).



Hình 16-10: Sơ đồ

Scaled Dot-Product và Multi-Head

4.9 Tổng kết: Tại sao Transformer mạnh đến vậy?

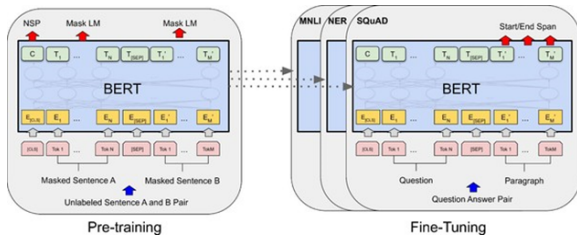
- **Cơ chế song song (Parallelization):** Huấn luyện nhanh hơn RNN gấp chục lần trên TPU/GPU.
- **Ngữ cảnh toàn cục $O(1)$:** Một từ cuối văn bản có thể liên kết trực tiếp với từ đầu văn bản mà không bị suy giảm thông tin.
- **Khả năng mở rộng không giới hạn (Scalability):** Càng thêm dữ liệu, càng tăng tham số (từ triệu lên hàng trăm tỷ), mô hình càng thông minh đột biến.

5.1 Các Họ Mô Hình Khổng lồ (Foundation Models)

- Sự ra đời của Transformer chia nhánh phát triển AI thành 3 họ chính:
- **Encoder-only (Họ BERT):** Hiểu văn bản xuất sắc. (Phân loại, trích xuất thực thể).
- **Decoder-only (Họ GPT):** Sinh văn bản xuất sắc. (Chatbot, tạo code, viết báo).
- **Encoder-Decoder (Họ T5, BART):** Cân bằng cả hai. (Dịch thuật, tóm tắt tự động).

5.2 Kiến trúc BERT (Bi-directional Encoder)

- **BERT (Google, 2018):** Chỉ sử dụng phần Encoder.
- Kỹ thuật Masked Language Model: Che ngẫu nhiên 15% từ và ép mô hình phải điền vào chỗ trống bằng cách nhìn cả trước và sau (2 chiều - Bi-directional).
- Ứng dụng mạnh mẽ để tinh chỉnh (Fine-tuning) cho các tác vụ NLP hẹp.



Hình 16-11: Cấu trúc của BERT để huấn luyện đa nhiệm

5.3 Vision Transformers (ViT) - Attention trong Computer Vision

- Kỹ nguyên Transformer không chỉ dừng ở NLP.
- **Cách làm:** Cắt ảnh lớn thành lưới các ô nhỏ (patches), chải phẳng và coi mỗi ô như một "từ".
- Dùng Transformer phân tích hình ảnh, đạt độ chính xác đánh bại CNN truyền thống (ResNet) trên các tập dữ liệu lớn.



Hình 16-12: Sự tập trung hình ảnh của mô hình Attention

5.4 Thư viện Transformers của Hugging Face

- **Hugging Face** là công ty cung cấp "GitHub cho Machine Learning", sở hữu kho Model Hub lớn nhất thế giới.
- Cung cấp các công cụ (Pipeline) mã nguồn mở để tải và sử dụng các mô hình khổng lồ dễ dàng.
- Giải phóng lập trình viên khỏi việc phải tự huấn luyện mô hình từ đầu, tiết kiệm hàng triệu đô la tiền thuê máy chủ.

5.5 Ví dụ: Tích hợp AI bằng Hugging Face Pipeline

- Chỉ với 4 dòng code, bạn đã đưa được mô hình State-of-the-Art vào ứng dụng:

```
from transformers import pipeline

# Khởi tạo pipeline phân tích cảm xúc
classifier = pipeline("sentiment-analysis")

# Dự đoán (Tu động tại mô hình về nếu chưa có)
result = classifier("I really loved this movie, it was fantastic!")
print(result)
# Kết quả: [{'label': 'POSITIVE', 'score': 0.9998}]
```

Tổng Kết Chương 16

- **RNN / GRU**: Nền tảng cốt lõi của xử lý dữ liệu tuần tự, nhưng bị giới hạn bởi nút thắt cổ chai và khó song song hóa.
- **Encoder-Decoder**: Kiến trúc vàng cho dịch máy, tạo tiền đề cho các mô hình Seq2seq.
- **Cơ chế Attention**: Cuộc cách mạng giải quyết bài toán bộ nhớ dài hạn, cho phép mô hình nhìn toàn cảnh.
- **Transformer**: Kẻ thay đổi cuộc chơi, khai sinh kỷ nguyên mô hình nền tảng (Foundation Models) và LLM.
- **Hugging Face**: Cầu nối đưa AI tiên tiến đến tay mọi lập trình viên.

HỎI & ĐÁP